📘 Testing con Repositories

🎯 Objetivos

- Entender por qué Repository facilita el testing
- Crear Fake Repositories para tests
- Testear Services sin base de datos
- Aplicar inyección de dependencias para tests

🔍 El Problema: Tests con Base de Datos

Sin Repository Pattern, testear services requiere una BD real:

```
# ❌ Test que depende de base de datos
def test_create_task():
    # Necesita BD configurada
    db = SessionLocal()

    # Necesita datos previos
    user = User(name="Test", email="test@test.com")
    db.add(user)
    db.commit()

    # Test real
    service = TaskService(db)
    task = service.create_task(TaskCreate(title="Test", user_id=user.id))

    assert task.title == "Test"

    # Limpiar
    db.delete(task)
    db.delete(user)
    db.commit()
    db.close()
```

Problemas

Problema	Impacto
Lento	BD es I/O, tests lentos
Frágil	Depende de estado de BD
Complejo	Setup y teardown
No aislado	Tests pueden interferir

✅ Solución: Fake Repositories

Un **Fake Repository** simula el comportamiento del repositorio real usando estructuras en memoria:

```
# tests/fakes.py
from models import Task, User
```

```python
class FakeTaskRepository:
    """Repositorio falso que usa diccionario en memoria"""

    def __init__(self):
        self.tasks: dict[int, Task] = {}
        self._next_id = 1

    def get_by_id(self, task_id: int) -> Task | None:
        return self.tasks.get(task_id)

    def get_all(self, skip: int = 0, limit: int = 100) -> list[Task]:
        tasks = list(self.tasks.values())
        return tasks[skip:skip + limit]

    def add(self, task: Task) -> Task:
        task.id = self._next_id
        self.tasks[task.id] = task
        self._next_id += 1
        return task

    def update(self, task: Task) -> Task:
        self.tasks[task.id] = task
        return task

    def delete(self, task: Task) -> None:
        if task.id in self.tasks:
            del self.tasks[task.id]

    # Métodos específicos
    def get_by_user(self, user_id: int) -> list[Task]:
        return [t for t in self.tasks.values() if t.user_id == user_id]

    def get_pending(self) -> list[Task]:
        return [t for t in self.tasks.values() if t.status != TaskStatus.DONE]


class FakeUserRepository:
    """Repositorio falso para User"""

    def __init__(self):
        self.users: dict[int, User] = {}
        self._next_id = 1

    def get_by_id(self, user_id: int) -> User | None:
        return self.users.get(user_id)

    def get_by_email(self, email: str) -> User | None:
        for user in self.users.values():
            if user.email == email:
                return user
        return None

    def add(self, user: User) -> User:
        user.id = self._next_id
        self.users[user.id] = user
```

```
        self._next_id += 1
        return user
```

🧪 Tests con Fakes

Test de Service

```python
# tests/test_task_service.py
import pytest
from datetime import date

from models import User, Task, TaskStatus
from schemas import TaskCreate
from services import TaskService
from exceptions import NotFoundError
from tests.fakes import FakeTaskRepository, FakeUserRepository


class TestTaskService:
    """Tests para TaskService usando fakes"""

    def setup_method(self):
        """Setup antes de cada test"""
        self.task_repo = FakeTaskRepository()
        self.user_repo = FakeUserRepository()

        # Usuario de prueba
        self.test_user = User(name="John", email="john@test.com")
        self.user_repo.add(self.test_user)

        # Service con fakes
        self.service = TaskService(
            task_repo=self.task_repo,
            user_repo=self.user_repo
        )

    def test_create_task_success(self):
        """Crear task con usuario válido"""
        data = TaskCreate(
            title="Test Task",
            description="Description",
            user_id=self.test_user.id
        )

        task = self.service.create_task(data)

        assert task.id == 1
        assert task.title == "Test Task"
        assert task.user_id == self.test_user.id
        assert task.status == TaskStatus.TODO

    def test_create_task_user_not_found(self):
        """Error si usuario no existe"""
        data = TaskCreate(
```

```
            title="Test",
            user_id=999  # No existe
        )

        with pytest.raises(NotFoundError) as exc:
            self.service.create_task(data)

        assert "User 999 not found" in str(exc.value)

    def test_complete_task_success(self):
        """Completar task existente"""
        # Crear task primero
        task = Task(title="Test", user_id=self.test_user.id)
        self.task_repo.add(task)

        completed = self.service.complete_task(task.id)

        assert completed.status == TaskStatus.DONE
        assert completed.completed_at is not None

    def test_complete_task_not_found(self):
        """Error si task no existe"""
        with pytest.raises(NotFoundError):
            self.service.complete_task(999)

    def test_complete_task_already_done(self):
        """Error si task ya está completada"""
        task = Task(
            title="Done",
            user_id=self.test_user.id,
            status=TaskStatus.DONE
        )
        self.task_repo.add(task)

        with pytest.raises(BusinessError) as exc:
            self.service.complete_task(task.id)

        assert "already completed" in str(exc.value)

    def test_get_user_tasks(self):
        """Obtener tasks de un usuario"""
        # Crear varias tasks
        for i in range(3):
            task = Task(title=f"Task {i}", user_id=self.test_user.id)
            self.task_repo.add(task)

        tasks = self.service.get_user_tasks(self.test_user.id)

        assert len(tasks) == 3
```

🎯 Fake Unit of Work

Para tests más completos, crea un Fake UoW:

```python
# tests/fakes.py
class FakeUnitOfWork:
    """Unit of Work falso para tests"""

    def __init__(self):
        self.users = FakeUserRepository()
        self.tasks = FakeTaskRepository()
        self.committed = False
        self.rolled_back = False

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type:
            self.rollback()

    def commit(self):
        self.committed = True

    def rollback(self):
        self.rolled_back = True


# Test usando Fake UoW
def test_transaction_commits():
    uow = FakeUnitOfWork()

    user = User(name="Test", email="test@test.com")
    uow.users.add(user)

    task = Task(title="Task", user_id=user.id)
    uow.tasks.add(task)

    uow.commit()

    assert uow.committed is True
    assert len(uow.tasks.tasks) == 1
```

📊 Comparación: Con y Sin Fakes

Sin Fakes (Integration Test)

```python
# Lento, requiere BD, complejo setup
@pytest.fixture
def db():
    engine = create_engine("sqlite:///:memory:")
    Base.metadata.create_all(engine)
    Session = sessionmaker(bind=engine)
    session = Session()
    yield session
    session.close()

def test_create_task(db):
```

```
    # Setup en BD real
    user = User(name="Test", email="test@test.com")
    db.add(user)
    db.commit()

    repo = TaskRepository(db)
    service = TaskService(task_repo=repo, user_repo=UserRepository(db))

    task = service.create_task(TaskCreate(title="Test", user_id=user.id))

    assert task.title == "Test"
```

Con Fakes (Unit Test)

```
# Rápido, sin BD, simple
def test_create_task():
    task_repo = FakeTaskRepository()
    user_repo = FakeUserRepository()

    user = User(name="Test", email="test@test.com")
    user_repo.add(user)

    service = TaskService(task_repo=task_repo, user_repo=user_repo)

    task = service.create_task(TaskCreate(title="Test", user_id=user.id))

    assert task.title == "Test"
```

🔧 Ejecutar Tests

```
# Ejecutar todos los tests
pytest tests/

# Ejecutar con verbose
pytest tests/ -v

# Ejecutar test específico
pytest tests/test_task_service.py::TestTaskService::test_create_task_success

# Ver cobertura
pytest tests/ --cov=services --cov-report=html
```

📋 Estructura de Tests

```
tests/
├── __init__.py
├── conftest.py             # Fixtures compartidos
├── fakes.py                # Fake repositories
├── test_task_service.py    # Tests de TaskService
├── test_user_service.py    # Tests de UserService
```

```
└── integration/              # Tests de integración (con BD)
    └── test_api.py
```

conftest.py

```python
import pytest
from tests.fakes import FakeTaskRepository, FakeUserRepository, FakeUnitOfWork


@pytest.fixture
def task_repo():
    return FakeTaskRepository()


@pytest.fixture
def user_repo():
    return FakeUserRepository()


@pytest.fixture
def uow():
    return FakeUnitOfWork()


@pytest.fixture
def test_user(user_repo):
    from models import User
    user = User(name="Test User", email="test@example.com")
    return user_repo.add(user)
```

✅ Beneficios del Testing con Repositories

Beneficio	Descripción
Velocidad	Tests en memoria son instantáneos
Aislamiento	Cada test tiene su propio estado
Simplicidad	No hay setup de BD
Confiabilidad	No depende de infraestructura
Enfoque	Testa lógica de negocio pura

✅ Checklist

- [ ] Sé crear Fake Repositories para tests
- [ ] Puedo testear Services sin base de datos
- [ ] Entiendo la diferencia entre unit test e integration test
- [ ] Sé estructurar tests con pytest

🎯 Resumen de la Semana

Esta semana aprendiste:

1. **Repository Pattern**: Abstrae acceso a datos
2. **BaseRepository**: Código genérico reutilizable
3. **Repositorios Específicos**: Queries particulares
4. **Unit of Work**: Transacciones coordinadas
5. **Testing**: Fake repositories para tests unitarios

La arquitectura actual:

```
Router → Service → Repository → Database
            ↓
        Unit of Work (coordina transacciones)
```

→ Siguiente semana: **MVC Completo** con todas las capas integradas